LAPORAN PRAKTIKUM

ALGORITMA PEMROGRAMAN 2

MODUL 5 – REKURSIF

NAMA : Raka Yulia Vegora

NIM : 109082500201

KELAS S1IF-13-06

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

1. Pengertian Rekursif

Rekursif adalah metode pemrograman di mana sebuah fungsi memanggil dirinya sendiri untuk menyelesaikan masalah yang dapat dipecah menjadi bagian yang lebih kecil dengan pola yang sama.

2. Struktur Dasar Rekursi

Fungsi rekursif memiliki dua komponen utama:

- **Base Case (Kondisi Dasar)**: kondisi untuk menghentikan rekursi agar tidak terjadi perulangan tanpa batas.
- **Recursive Case (Kondisi Rekursif)**: bagian yang memanggil fungsi itu sendiri dengan parameter yang lebih kecil.

3. Cara Kerja Rekursi

Fungsi akan terus memanggil dirinya sendiri hingga mencapai kondisi dasar. Setelah itu, proses berhenti dan hasil dikembalikan secara bertahap ke pemanggil sebelumnya (unwinding).

4. Contoh Rekursi

Contoh: faktorial

$$\mathrm{n}! = \mathrm{n} \times (\mathrm{n} - 1)! \text{ dengan} 0! = 1$$

5. Kelebihan Rekursi

- Kode lebih sederhana untuk masalah tertentu
- Cocok untuk struktur data seperti tree dan graf
- Mendukung metode divide and conquer

6. Kekurangan Rekursi

- Menggunakan memori lebih besar (stack)
- Berisiko stack overflow
- Bisa lebih lambat dibanding iterasi

B. Guided

1. barisbilangan.go

```go
package main
import "fmt"

func main(){
    var n int
    fmt.Scan(&n)
    baris(n)
}

func baris(bilangan int){
    if bilangan == 1 {
        fmt.Println(1)
    }else{
        fmt.Println(bilangan)
        baris(bilangan - 1)
    }
}
```

Penjelasan :

Program ini berfungsi untuk menerima sebuah input angka dari pengguna, kemudian angka tersebut diproses dengan memanggil fungsi baris. Fungsi ini bekerja secara rekursif, yaitu dengan mencetak nilai angka yang dimasukkan lalu memanggil dirinya sendiri dengan nilai yang dikurangi satu hingga mencapai angka 1. Saat nilai sudah sama dengan 1, program akan mencetak angka 1 dan proses berhenti. Dengan demikian, program akan menampilkan deretan angka dari nilai input hingga 1 secara menurun.

Output :

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\LENOVO\OneDrive\Documents\109082500201_Raka-Yulia-Vegora> go run "c:\Users\LENOVO\OneDrive
\Documents\109082500201_Raka-Yulia-Vegora\Week 4 - Modul 4\Guided\Guided-1\barisbilangan.go"
5
5
4
3
2
1
```

2. hasilpenjumlahan.go

```go
package main
import "fmt"

func main(){
    var n int
    fmt.Scan(&n)
    fmt.Println(penjumlahan(n))
}

func penjumlahan(n int) int {
    if n == 1 {
        return 1
        }else{
            return n + penjumlahan(n-1)
        }
}
```

Penjelasan :

Program berfungsi untuk menerima sebuah input angka dari pengguna, kemudian menghitung jumlah seluruh bilangan dari 1 sampai angka tersebut. Setelah pengguna memasukkan nilai n, program akan memanggil fungsi penjumlahan yang bekerja secara rekursif, yaitu dengan menjumlahkan nilai n dengan hasil pemanggilan fungsi yang sama menggunakan nilai n-1. Proses ini terus berulang hingga mencapai kondisi dasar saat n bernilai 1, di mana fungsi akan mengembalikan nilai 1. Hasil akhir dari proses tersebut kemudian ditampilkan sebagai jumlah total deret bilangan dari 1 sampai n.

Output :

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\LENOVO\OneDrive\Documents\109082500201_Raka-Yulia-Vegora> go run "c:\Users\LENOVO\OneDrive
\Documents\109082500201_Raka-Yulia-Vegora\Week 4 - Modul 4\Guided\Guided-2\hasilpenjumlahan.go"
20
210
```

3. duapangkat.go

```go
package main
import "fmt"

func main(){
    var n int
    fmt.Scan(&n)
    fmt.Println(pangkat(n))
}

func pangkat(n int) int {
    if n == 0 {
        return 1
    }else{
        return 2 * pangkat(n-1)
    }
}
```

Penjelasan :

Program ini berfungsi untuk menerima sebuah input angka dari pengguna, kemudian menghitung hasil perpangkatan 2 dengan pangkat sesuai nilai input tersebut. Setelah pengguna memasukkan nilai n, program akan memanggil fungsi pangkat dan menampilkan hasilnya. Fungsi pangkat bekerja secara rekursif, yaitu dengan mengalikan angka 2 dengan hasil pemanggilan fungsi yang sama menggunakan nilai n-1. Proses ini terus berulang hingga mencapai kondisi dasar saat n bernilai 0, di mana fungsi akan mengembalikan nilai 1. Dengan demikian, program akan menghasilkan nilai 2 pangkat n (2^n) berdasarkan input yang diberikan.

Output :

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\LENOVO\OneDrive\Documents\109082500201_Raka-Yulia-Vegora> go run "c:\Users\LENOVO\OneDrive
\Documents\109082500201_Raka-Yulia-Vegora\Week 4 - Modul 4\Guided\Guided-3\duapangkat.go"
25
33554432
```

C. Unguided

1. Soal Unguided 1

[Deret fibonacci adalah sebuah deret dengan nilai suku ke-0 dan ke-1 adalah 0 dan 1, dan nilai suku ke-n selanjutnya adalah hasil penjumlahan dua suku sebelumnya. Secara umum dapat diformulasikan $S_n = S_{n-1} + S_{n-2}$. Berikut ini adalah contoh nilai deret fibonacci hingga suku ke-10. Buatlah program yang mengimplementasikan fungsi rekursif pada deret Fibonacci tersebut.

Jawaban :

```go
package main
import "fmt"

func fibonacci(n int) int {
    if n == 0 {
        return 0
    } else if n == 1 {
        return 1
    } else {
        return fibonacci(n-1) + fibonacci(n-2)
    }
}

func main() {
    var n int
    fmt.Print("Masukkan nilai n: ")
    fmt.Scan(&n)

    fmt.Println("Deret Fibonacci:")
    for i := 0; i <= n; i++ {
        fmt.Print(fibonacci(i), " ")
    }
}
```

Penjelasan :

Program ini digunakan untuk menampilkan **deret Fibonacci** menggunakan bahasa Go. Program meminta pengguna memasukkan nilai **n**, kemudian fungsi fibonacci() menghitung deret Fibonacci secara rekursif (memanggil dirinya sendiri). Setelah itu, pada fungsi main()

program melakukan perulangan dari 0 sampai n dan menampilkan hasil deret Fibonacci ke layar.

Output :

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\LENOVO\OneDrive\Documents\109082500201_Raka-Yulia-Vegora> go run "c:\Users\LENOVO\OneDrive
\Documents\109082500201_Raka-Yulia-Vegora\Week 4 - Modul 4\Unguided\unguided-no1\tempCodeRunnerFile.go
"
Masukkan nilai n: 10
Deret Fibonacci:
0 1 1 2 3 5 8 13 21 34 55
```

2. Soal Unguided 2

   Buatlah sebuah program yang digunakan untuk menampilkan pola bintang berikut ini dengan menggunakan fungsi rekursif. N adalah masukan dari user.

   Jawaban :

```go
package main
import "fmt"

func bintang(n int) {
    if n == 0 {
        return
    }

    bintang(n - 1)

    for i := 0; i < n; i++ {
        fmt.Print("*")
    }
    fmt.Println()
}

func main() {
    var n int

    fmt.Print("Masukkan nilai N: ")
    fmt.Scan(&n)

    fmt.Println("Pola Bintang:")
    bintang(n)
}
```

   Penjelasan :

Program tersebut digunakan untuk menampilkan pola bintang segitiga dengan metode rekursi. Program diawali dengan penggunaan package main dan fmt untuk input dan output.

Fungsi bintang(n int) adalah fungsi rekursif yang memiliki kondisi dasar saat n == 0 untuk menghentikan proses. Jika belum terpenuhi, fungsi akan memanggil dirinya sendiri dengan n-1, kemudian mencetak bintang sebanyak n menggunakan perulangan.

Pada fungsi main, pengguna diminta memasukkan nilai n, lalu fungsi bintang(n) dipanggil untuk menampilkan pola.

Secara keseluruhan, program bekerja dengan rekursi dan menghasilkan pola bintang dari kecil ke besar sesuai nilai input.

Output :

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

\LENOVO\OneDrive\Documents\109082500201_Raka-Yulia-Vegora\unguided-no2\polabintang.go"
Masukkan nilai N: 5
Pola Bintang:
*
**
***
****
*****
PS C:\Users\LENOVO\OneDrive\Documents\109082500201_Raka-Yulia-Vegora> go run "c:\Users
\LENOVO\OneDrive\Documents\109082500201_Raka-Yulia-Vegora\Week 4 - Modul 4\Unguided\un
guided-no2\polabintang.go"
Masukkan nilai N: 1
Pola Bintang:
*
PS C:\Users\LENOVO\OneDrive\Documents\109082500201_Raka-Yulia-Vegora> go run "c:\Users
\LENOVO\OneDrive\Documents\109082500201_Raka-Yulia-Vegora\Week 4 - Modul 4\Unguided\un
guided-no2\polabintang.go"
Masukkan nilai N: 3
Pola Bintang:
*
**
***
```

3. Soal Unguided 3

   Buatlah program yang mengimplementasikan rekursif untuk menampilkan faktor bilangan dari suatu N, atau bilangan yang apa saja yang habis membagi N.

   Jawaban :

```go
package main
import "fmt"

func faktor(n int, i int) {
    if i > n {
        return
    }

    if n%i == 0 {
        fmt.Print(i, " ")
    }

    faktor(n, i+1)
}

func main() {
    var n int

    fmt.Print("Masukkan bilangan N: ")
    fmt.Scan(&n)

    fmt.Println("Faktor dari", n, "adalah:")
    faktor(n, 1)
}
```

   Penjelasan :

Program tersebut digunakan untuk menampilkan faktor dari suatu bilangan **N** menggunakan rekursif. Fungsi faktor(n, i) berfungsi mengecek setiap bilangan dari 1 sampai N, jika n % i == 0 maka bilangan tersebut adalah faktor dan ditampilkan, lalu fungsi memanggil dirinya sendiri hingga selesai. Pada fungsi main(), program meminta input N dari pengguna menggunakan fmt.Scan, kemudian memanggil fungsi faktor(n, 1) untuk menampilkan semua faktor dari bilangan N.

Output :

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\LENOVO\OneDrive\Documents\109082500201_Raka-Yulia-Vegora> go run "c:\Users\LENOVO\OneDrive
\Documents\109082500201_Raka-Yulia-Vegora\Week 4 - Modul 4\Unguided\unguided-no3\faktorbilangan.go"
Masukkan bilangan N: 5
Faktor dari 5 adalah:
1 5
```

D. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa rekursif merupakan metode pemrograman yang efektif untuk menyelesaikan masalah dengan cara memanggil fungsi itu sendiri hingga mencapai kondisi dasar. Penerapan rekursif dapat digunakan dalam berbagai kasus seperti deret bilangan, penjumlahan, perpangkatan, deret Fibonacci, pola bintang, serta pencarian faktor bilangan.

Melalui praktikum ini, dapat dipahami bahwa rekursi mempermudah penulisan program untuk masalah tertentu karena struktur kodenya lebih sederhana. Namun, penggunaan rekursi juga memiliki kelemahan seperti penggunaan memori yang lebih besar dan risiko terjadinya stack overflow jika tidak dirancang dengan baik.

Dengan demikian, pemahaman konsep rekursif sangat penting agar dapat digunakan secara tepat dan efisien dalam pemrograman.